# OBJECT-ORIENTED PROGRAMMING

15-Page Study Notes

Beginner-friendly notes with Python and C++ examples

**Topics included**

Classes and objects • constructors and destructors • self and this • encapsulation • abstraction • inheritance • polymorphism • exception handling • practice questions

**How to study:** Read the concept, trace the example, then try changing the code yourself. The examples are educational and can be copied into a Python or C++ editor.

Prepared as a compact revision guide

# 1. Introduction to OOP

**Object-Oriented Programming (OOP)** is a programming approach that organizes code around objects. An object combines data (attributes) and behavior (methods). OOP helps represent real-world entities and can make larger programs easier to organize, extend, test, and maintain.

## Procedural programming vs OOP

In procedural programming, code is commonly organized around functions and steps. In OOP, related data and operations are grouped into classes and objects. Both approaches are useful; the right choice depends on the problem.

## Core vocabulary

**Class:** a blueprint that defines attributes and methods.
**Object:** an instance created from a class.
**Attribute:** data stored on an object or class.
**Method:** a function associated with a class.
**State:** an object's current data.
**Behavior:** what the object can do.

Example: a Student object may store a name and age, and provide a method to display them.

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print(f"I'm {self.name}, age {self.age}")

student = Student("Tayyeb", 22)
student.introduce()
```

Output: I'm Tayyeb, age 22

## Benefits of OOP

• Modularity: keep related code together.
• Reuse: reuse or extend existing classes.
• Maintainability: changes can be localized.
• Modeling: classes can represent concepts such as Student, Product, or BankAccount.
• Encapsulation: protect object state through a clear interface.

Important: OOP is a tool, not a rule that every program must follow. Small scripts may be simpler with ordinary functions.

# 2. Classes and Objects

A class defines the structure and behavior that its objects share. Each object can hold its own attribute values. Creating an object is often called instantiation.

## Python example

```
class Car:
    def __init__(self, brand, year):
        self.brand = brand
        self.year = year

    def show_info(self):
        print(self.brand, self.year)

car1 = Car("Toyota", 2022)
car2 = Car("Honda", 2024)

car1.show_info()
car2.show_info()
```

Output:
Toyota 2022
Honda 2024

Here, `Car` is the class; `car1` and `car2` are separate objects. Their `brand` and `year` values are independent.

## C++ example

```
#include <iostream>
#include <string>
using namespace std;

class Car {
public:
    string brand;
    int year;

    void show_info() {
        cout << brand << " " << year << endl;
    }
};

int main() {
    Car car1;
    car1.brand = "Toyota";
    car1.year = 2022;
    car1.show_info();
    return 0;
}
```

## Accessing members

In Python, use dot notation such as `car1.brand`. In C++, dot notation accesses members of an ordinary object; pointers commonly use `->`. Access control such as `public` and `private` is built into C++. Python instead commonly uses naming conventions and properties.

# 3. Constructors and Initialization

A constructor or initializer sets up an object when it is created. In Python, `__init__()` initializes an already-created instance. In C++, a constructor has the same name as the class and no return type.

## Python: __init__

```
class Student:
    def __init__(self, name, age=18):
        self.name = name
        self.age = age

s1 = Student("Tayyeb", 22)
s2 = Student("Ali")
print(s1.name, s1.age)
print(s2.name, s2.age)
```

Output:
Tayyeb 22
Ali 18

The `self` parameter refers to the current instance. The name `self` is a strong Python convention, not a reserved keyword. Default values allow the caller to omit an argument.

## C++: constructor

```
#include <iostream>
#include <string>
using namespace std;

class Student {
public:
    string name;
    int age;

    Student(string n, int a) : name(n), age(a) {}
};

int main() {
    Student s1("Tayyeb", 22);
    cout << s1.name << " " << s1.age << endl;
}
```

## Common constructor patterns

**Default:** takes no arguments.
**Parameterized:** accepts values to initialize an object.
**Copy constructor (C++):** initializes an object from another object of the same type.
**Move constructor (C++):** can transfer resources from a movable object.

Python does not support multiple methods named `__init__` as overloads in the same class. Defaults or alternative class methods are common ways to support different creation patterns.

# 4. Destructors and Object Lifetime

A destructor or finalization method relates to cleanup when an object is no longer needed. The details differ importantly between C++ and Python.

## C++ destructor

```
#include <iostream>
using namespace std;

class Demo {
public:
    Demo() { cout << "Created" << endl; }
    ~Demo() { cout << "Destroyed" << endl; }
};

int main() {
    Demo obj;
    cout << "Using object" << endl;
    return 0;
}
```

Typical output:
Created
Using object
Destroyed

For a local C++ object, the destructor runs when its lifetime ends. Destructors are important for resource management. Modern C++ commonly uses RAII: tie resource ownership to object lifetime.

## Python: __del__

```
class Demo:
    def __init__(self):
        print("Created")

    def __del__(self):
        print("Finalizing object")

obj = Demo()
del obj
```

Python's `del obj` removes a reference; it does not guarantee immediate destruction if other references remain. The timing of `__del__()` is not generally guaranteed, so do not rely on it for critical cleanup.

## Recommended cleanup

Use `with open(...)` for files in Python and context managers for other supported resources. In C++, use RAII types such as `std::string`, `std::vector`, and smart pointers instead of manually managing resources where possible.

# 5. The Meaning of self and this

In Python, `self` is the conventional first parameter of an instance method. It refers to the instance on which the method is called. In C++, `this` is a pointer to the current object inside a non-static member function.

## Python example

```
class Counter:
    def __init__(self, value):
        self.value = value

    def increment(self):
        self.value += 1

c1 = Counter(0)
c2 = Counter(10)
c1.increment()
print(c1.value)  # 1
print(c2.value)  # 10
```

Each instance has its own value. When `c1.increment()` is called, Python supplies `c1` as the method's first argument automatically.

## C++ example

```
class Counter {
public:
    int value;

    Counter(int value) {
        this->value = value;
    }
};
```

The expression `this->value` refers to the current object's member, while the constructor parameter is also named `value`. A C++ constructor can often use an initializer list instead: `Counter(int value) : value(value) {}`.

## Key difference

Python's `self` is an ordinary parameter name by convention. C++'s `this` is a language-provided pointer and cannot be renamed. Both help a method refer to the current object.

# 6. Encapsulation

Encapsulation groups data and methods in a class and controls how the object's internal state is accessed or changed. It helps keep an object valid by providing a clear public interface.

## Python example using a property

```
class BankAccount:
    def __init__(self, balance=0):
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Amount must be positive")
        self._balance += amount

account = BankAccount(100)
account.deposit(50)
print(account.balance)  # 150
```

A leading underscore, as in `_balance`, signals that an attribute is intended for internal use. It is a convention, not strict access enforcement. A property gives a controlled way to read a value.

## C++ access modifiers

**public:** accessible through the public interface.
**private:** accessible only to members and friends of the class.
**protected:** accessible to the class and its derived classes, subject to C++ rules.

```
class BankAccount {
private:
    double balance;

public:
    BankAccount(double initial) : balance(initial) {}

    void deposit(double amount) {
        if (amount > 0) balance += amount;
    }

    double get_balance() const {
        return balance;
    }
};
```

## Why it matters

Encapsulation prevents arbitrary code from changing data in ways that break assumptions. Validate inputs at the object's boundary and expose operations that make invalid states harder to create.

# 7. Abstraction

Abstraction exposes essential operations while hiding implementation details. A caller needs to know what an object can do, not every internal step used to do it.

## Python abstract base class

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass

class Dog(Animal):
    def sound(self):
        return "Bark"

dog = Dog()
print(dog.sound())
```

Output: Bark

Because `Animal` has an unimplemented abstract method, Python does not allow it to be instantiated directly. A concrete subclass must implement the abstract method.

## C++ abstract class

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void sound() = 0;
    virtual ~Animal() = default;
};

class Dog : public Animal {
public:
    void sound() override {
        cout << "Bark" << endl;
    }
};
```

The `= 0` syntax declares a pure virtual function. A class with at least one pure virtual function is abstract and cannot be instantiated directly.

## Abstraction vs encapsulation

Abstraction focuses on the essential interface and hides complexity. Encapsulation groups state and behavior and controls access to internal data. They often work together, but they are not identical.

# 8. Inheritance

Inheritance allows a new class to derive from an existing class. The derived class can reuse or extend behavior from the base class. Use inheritance when there is a genuine 'is-a' relationship.

## Python example

```
class Animal:
    def eat(self):
        print("Eating...")

class Dog(Animal):
    def bark(self):
        print("Woof!")

dog = Dog()
dog.eat()   # inherited method
dog.bark()
```

## C++ example

```
#include <iostream>
using namespace std;

class Animal {
public:
    void eat() { cout << "Eating..." << endl; }
    virtual ~Animal() = default;
};

class Dog : public Animal {
public:
    void bark() { cout << "Woof!" << endl; }
};
```

## Common inheritance forms

**Single:** one derived class inherits from one base class.
**Multilevel:** a class derives from a class that is itself derived.
**Hierarchical:** multiple derived classes share a base class.
**Multiple:** one class inherits from multiple base classes; supported by both Python and C++, with language-specific details.

## Method overriding

A subclass can provide its own implementation of an inherited method. In C++, use `virtual` in the base class for runtime dispatch and `override` in the derived class to make intent clear. In Python, a subclass method with the same name overrides the inherited method.

## Design note

Prefer composition (a 'has-a' relationship) when an object should contain or use another object rather than be a specialized version of it.

# 9. Polymorphism

Polymorphism means 'many forms': the same interface or operation can produce different behavior depending on the object or arguments.

## Runtime polymorphism in Python

```
class Dog:
    def sound(self):
        return "Bark"

class Cat:
    def sound(self):
        return "Meow"

def make_sound(animal):
    print(animal.sound())

make_sound(Dog())
make_sound(Cat())
```

Output:
Bark
Meow

Python uses dynamic dispatch and duck typing: if an object supports the required method, a function can often use it without requiring a shared base class.

## Runtime polymorphism in C++

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void sound() { cout << "Animal sound" << endl; }
    virtual ~Animal() = default;
};

class Dog : public Animal {
public:
    void sound() override { cout << "Bark" << endl; }
};

int main() {
    Dog dog;
    Animal* animal = &dog;
    animal->sound();  // Bark
}
```

Because `sound()` is virtual, calling it through a base-class pointer dispatches to the derived object's implementation.

## Other forms

**Overriding:** a subclass replaces an inherited implementation.
**Overloading:** multiple functions or methods share a name but have different parameter lists (supported differently by languages).
**Operator overloading:** define how operators behave for a user-defined type; supported by C++ and by Python's special methods such as `__add__`.

# 10. Composition, Association, and Aggregation

Objects often work together. OOP relationships help describe whether one object owns another, merely uses it, or shares a reference to it.

**Association:** a general relationship between objects. For example, a teacher teaches students.
**Aggregation:** a whole-part relationship where the part can exist independently. For example, a team has players who can exist outside that team.
**Composition:** a stronger whole-part relationship where the whole manages the part's lifecycle by design.

## Python composition example

```
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        self.engine.start()
        print("Car ready")

car = Car()
car.start()
```

Output:
Engine started
Car ready

The Car delegates engine behavior to an Engine object. This is composition: a car has an engine. The exact ownership/lifetime guarantee depends on the design and references in the language.

## Why composition can be better than inheritance

• Components can be replaced independently.
• Classes avoid inheriting behavior they do not need.
• Dependencies can be injected for testing.
• Large systems can be built from smaller cooperating objects.

## Example from AI engineering

A RAG application might have separate objects for a document loader, text splitter, embedding model, vector store, retriever, and answer generator. A pipeline object can coordinate them rather than putting every responsibility into one enormous class.

# 11. Methods, Class Attributes, and Static Methods

Python supports instance methods, class methods, and static methods. Choose based on what data the operation needs.

## Python example

```
class Employee:
    company = "Example Ltd"  # class attribute

    def __init__(self, name):
        self.name = name     # instance attribute

    def introduce(self):
        return f"I'm {self.name}"

    @classmethod
    def change_company(cls, name):
        cls.company = name

    @staticmethod
    def is_valid_name(name):
        return isinstance(name, str) and bool(name.strip())

e = Employee("Tayyeb")
print(e.introduce())
print(Employee.is_valid_name("Ali"))
Employee.change_company("New Company")
print(Employee.company)
```

Instance method: receives `self` and works with instance data.
**Class method:** receives `cls` and commonly works with class-level state or alternate constructors.
**Static method:** receives neither automatically; useful for a related utility function.

Class attributes are shared through the class unless shadowed by an instance attribute. Instance attributes belong to each object.

## C++ note

C++ has static data members and static member functions. A static member function has no `this` pointer and cannot directly access a particular object's non-static members.

## Useful design habit

Keep each class focused. If a class handles unrelated jobs—database access, user interface, model inference, and logging—it may be doing too much. Split responsibilities into small, cohesive classes.

# 12. Exception Handling in OOP

Exceptions represent errors or unusual conditions that interrupt normal control flow. Exception handling lets a program respond, recover where possible, or report a useful error.

## Python: try / except / else / finally

```
try:
    number = int(input("Enter an integer: "))
    result = 100 / number
except ValueError:
    print("Please enter a valid integer.")
except ZeroDivisionError:
    print("Number cannot be zero.")
else:
    print("Result:", result)
finally:
    print("Finished.")
```

**try:** code that may raise an exception.
**except:** handles a matching exception.
**else:** runs when the try block completes without an exception.
**finally:** runs during normal exception handling whether an exception occurred or not, barring process termination or similar circumstances.

## C++: try / throw / catch

```
#include <iostream>
#include <stdexcept>
using namespace std;

int divide(int a, int b) {
    if (b == 0) {
        throw invalid_argument("Division by zero");
    }
    return a / b;
}

int main() {
    try {
        cout << divide(10, 0) << endl;
    } catch (const exception& e) {
        cout << "Error: " << e.what() << endl;
    }
}
```

C++ integer division by zero should be checked explicitly; it does not automatically throw a standard exception. Catch exceptions by reference when appropriate, and prefer specific exception types.

## Good practices

Catch only errors you can handle meaningfully. Avoid silently ignoring exceptions. Provide useful context in error messages, and use resource-management patterns such as context managers in Python and RAII in C++.

# 13. SOLID Principles and Good Design

SOLID is a set of five design principles often used to make object-oriented code easier to maintain. They are guidelines rather than absolute laws.

**S — Single Responsibility Principle:** a class should have one main reason to change. Example: keep report formatting separate from database storage.

**O — Open/Closed Principle:** software entities should be open to extension but closed to repeated modification where practical. Example: add a new payment strategy through a common interface.

**L — Liskov Substitution Principle:** a subtype should be usable wherever its base type is expected without breaking required behavior.

**I — Interface Segregation Principle:** prefer small, focused interfaces over one large interface that forces clients to depend on unused operations.

**D — Dependency Inversion Principle:** high-level logic should depend on abstractions rather than tightly coupling itself to concrete implementation details.

## Simple strategy example

```
class EmailSender:
    def send(self, message):
        print("Email:", message)

class Notifier:
    def __init__(self, sender):
        self.sender = sender

    def notify(self, message):
        self.sender.send(message)

notifier = Notifier(EmailSender())
notifier.notify("Your report is ready")
```

The Notifier receives a sender dependency rather than creating one internally. That makes it easier to supply a different sender or a test double. In a larger design, a formal interface or protocol can clarify the contract.

## OOP best practices

• Use meaningful class and method names.
• Keep classes small and cohesive.
• Prefer composition when inheritance does not represent a true 'is-a' relationship.
• Validate data at clear boundaries.
• Write tests for public behavior.
• Avoid unnecessary abstraction; start simple and refactor when a real need appears.

# 14. Revision Sheet and Practice Questions

## Quick revision

**Class:** blueprint. **Object:** instance. **Attribute:** data. **Method:** behavior.
**Constructor/initializer:** sets up an object (`__init__` in Python; class-named constructor in C++).
**Destructor/finalization:** cleanup at end of lifetime or finalization (`~ClassName` in C++; `__del__` in Python, whose timing is not guaranteed).
**Encapsulation:** group state and behavior and control access.
**Abstraction:** expose essential behavior, hide implementation complexity.
**Inheritance:** derive a class from another.
**Polymorphism:** same interface, different behavior.
**Exception handling:** Python uses `try/except`; C++ uses `try/catch` and can raise with `throw`.

## Practice questions

1. What is the difference between a class and an object?
2. Why does Python instance methods use `self`?
3. What is the difference between a Python initializer and a C++ constructor?
4. Why should Python code not rely on the exact timing of `__del__`?
5. Explain encapsulation with a bank account example.
6. How does an abstract method help define a contract?
7. What is the difference between inheritance and composition?
8. Explain overriding and runtime polymorphism.
9. What are `try`, `except`, `throw`, and `catch`?
10. Name the five SOLID principles.

## Mini task

Create a `Product` class with name and price attributes. Add a method that displays the product and a method that applies a discount only if the discount is between 0 and 100 percent. Then create two products and test valid and invalid discounts.

## Suggested answers

1. A class defines structure; an object is an instance. 2. `self` refers to the current instance. 3. Python's `__init__` initializes an instance; a C++ constructor participates in object construction. 4. Python finalization timing depends on references and runtime behavior. 5. Keep balance private or controlled and expose deposit/withdraw operations. 6. It requires concrete subclasses to implement a method. 7. Inheritance is 'is-a'; composition is 'has-a'. 8. Overriding changes inherited behavior; polymorphism dispatches through a shared interface. 9. Python handles with `except`; C++ handles with `catch` and can signal via `throw`. 10. SRP, OCP, LSP, ISP, DIP.

End of notes — revise the examples by typing and running them yourself.